



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 10: Problemas clásicos de sincronización

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 3 de octubre de 2026

Índice general

Índice de figuras	III
Índice de cuadros	IV
1. Problema productor-consumidor	1
1.1. Entradas	1
1.2. Procesamiento	2
1.3. Prueba de escritorio	3
1.4. Salidas y evidencia	4
1.5. Conceptos utilizados	4
2. Problema lectores-escriitores	5
2.1. Entradas	5
2.2. Procesamiento	6
2.3. Prueba de escritorio	6
2.4. Salidas y evidencia	7
2.5. Conceptos utilizados	10
3. Conclusión	11
Referencias	12

Índice de figuras

1.1. Código Java y salidas del problema productor-consumidor. .	4
2.1. Código Java de los lectores y escritores, parte superior. . . .	8
2.2. Código Java de los lectores y escritores, parte inferior. . . .	9
2.3. Creación de los hilos y resultados de las ejecuciones del problema lectores-escritores.	10

Índice de cuadros

1.1.	Datos iniciales del problema productor-consumidor.	2
1.2.	Prueba de escritorio del búfer circular.	3
2.1.	Datos iniciales del problema lectores-escritores.	5
2.2.	Prueba de escritorio del problema lectores-escritores.	7

1. Problema

productor-consumidor

Instrucción. Implementar un búfer circular compartido con capacidad para cinco elementos. El programa debe incluir un hilo productor que genere y deposite diez elementos, un hilo consumidor que retire y procese los diez elementos, un semáforo binario `mutex`, un semáforo contador para los espacios disponibles y otro para los elementos disponibles. Los índices `inBuf` y `outBuf` deben controlar el arreglo circular. El productor debe esperar si el búfer está lleno y el consumidor debe esperar si está vacío.

1.1 Entradas

El programa utiliza un arreglo llamado `buffer` con cinco posiciones. El productor genera los elementos del 1 al 10 y el consumidor retira esos mismos diez elementos. Los dos hilos comparten el arreglo, los índices `inBuf` y `outBuf`, y tres semáforos:

Cuadro 1.1: Datos iniciales del problema productor-consumidor.

Elemento	Valor inicial	Función
mutex	1	Protege el acceso al búfer. Solo un hilo puede modificarlo a la vez.
espacios	5	Cuenta los lugares libres del búfer.
elementos	0	Cuenta los elementos disponibles para el consumidor.
inBuf y outBuf	0	Indican la posición donde se deposita o se retira el siguiente elemento.

Al inicio, el búfer está vacío. Por eso hay cinco espacios disponibles y no hay elementos que el consumidor pueda retirar.

1.2 Procesamiento

El productor solicita primero un espacio mediante el semáforo espacios. Después solicita mutex para entrar al búfer, deposita el elemento en la posición inBuf y avanza el índice con la operación circular $(inBuf + 1) \% 5$. Al terminar, libera mutex y aumenta el semáforo elementos.

El consumidor realiza el proceso contrario. Primero solicita un elemento con elementos.acquire() y después solicita mutex. Retira el valor de la posición outBuf, avanza ese índice de forma circular, libera mutex y aumenta espacios.

Los semáforos controlan dos situaciones distintas. mutex evita que productor y consumidor modifiquen el arreglo al mismo tiempo. Los semáforos espacios y elementos coordinan la condición del búfer: el productor se bloquea cuando no hay espacio y el consumidor se bloquea cuando no hay elementos.

1.3 Prueba de escritorio

La siguiente secuencia muestra una posible ejecución. Los valores de los semáforos se escriben después de cada acción; durante la sección crítica, mutex toma temporalmente el valor 0.

Cuadro 1.2: Prueba de escritorio del búfer circular.

Paso	Acción	Semáforos	Índices	Búfer y estado
0	Estado inicial	1/5/0	0/0	[_,_,_,_,_]. Ambos hilos están activos.
1	Productor deposita 1	1/4/1	1/0	[1,_,_,_,_]. El consumidor puede continuar.
2	Productor deposita 2	1/3/2	2/0	[1,2,_,_,_].
3	Productor deposita 3	1/2/3	3/0	[1,2,3,_,_].
4	Productor deposita 4	1/1/4	4/0	[1,2,3,4,_].
5	Productor deposita 5	1/0/5	0/0	[1,2,3,4,5]. El búfer está lleno.
6	Productor intenta depositar 6	1/0/5	0/0	El productor se bloquea al solicitar un espacio.
7	Consumidor retira 1	1/1/4	0/1	[_ ,2,3,4,5]. Se libera un espacio y el productor puede continuar.
8	Productor deposita 6	1/0/5	1/1	[6,2,3,4,5].
9	Consumidor retira 2	1/1/4	1/2	[6,_,3,4,5].

1.4 Salidas y evidencia

La captura muestra el código Java del productor-consumidor y varias ejecuciones del programa. En cada ejecución aparecen diez mensajes del productor y diez mensajes del consumidor. Los índices recorren las posiciones de 0 a 4 y vuelven a 0, lo que comprueba que el arreglo se utiliza como un búfer circular.

```

1 public class Ejercicio01ProductorConsumidor {
2
3     private static final int[] buffer = new int[5];
4     private static int inBuf = 0;
5     private static int outBuf = 0;
6
7     private static final Semaphore mutex = new Semaphore(1);
8     private static final Semaphore espacios = new Semaphore(5);
9     private static final Semaphore elementos = new Semaphore(0);
10
11     private static void producir(int elemento) {
12         try {
13             espacios.acquire(); // espera si el buffer esta lleno
14             mutex.acquire(); // entra al buffer
15             buffer[inBuf] = elemento;
16             System.out.println("Productor deposita " + elemento + " en " + inBuf);
17             inBuf = (inBuf + 1) % 5;
18             mutex.release();
19             elementos.release(); // ahora hay un elemento mas
20         } catch (InterruptedException e) {
21             Thread.currentThread().interrupt();
22         }
23     }
24
25     private static void consumir() {
26         try {
27             elementos.acquire(); // espera si el buffer esta vacio
28             mutex.acquire(); // entra al buffer
29             int elemento = buffer[outBuf];
30             System.out.println("Consumidor retira " + elemento + " de " + outBuf);
31             outBuf = (outBuf + 1) % 5;
32             mutex.release();
33             espacios.release(); // ahora hay un espacio mas
34         } catch (InterruptedException e) {
35             Thread.currentThread().interrupt();
36         }
37     }
38
39     public static void main(String[] args) throws InterruptedException {
40         Thread productor = new Thread() -> {
41             for (int i = 1; i <= 10; i++) {
42                 producir(i);
43             }
44         };
45         Thread consumidor = new Thread() -> {
46             for (int i = 1; i <= 10; i++) {
47                 consumir(i);
48             }
49         };
50         productor.start();
51         consumidor.start();
52     }
53 }

```

Output (herf1 -/concurrente - P main ?):

```

Productor deposita 1 en 0
Productor deposita 2 en 1
Productor deposita 3 en 2
Productor deposita 4 en 3
Productor deposita 5 en 4
Consumidor retira 1 de 0
Consumidor retira 2 de 1
Consumidor retira 3 de 2
Consumidor retira 4 de 3
Consumidor retira 5 de 4
Productor deposita 6 en 0
Productor deposita 7 en 1
Productor deposita 8 en 2
Productor deposita 9 en 3
Productor deposita 10 en 4
Consumidor retira 6 de 0
Consumidor retira 7 de 1
Consumidor retira 8 de 2
Consumidor retira 9 de 3
Consumidor retira 10 de 4

```

Figura 1.1: Código Java y salidas del problema productor-consumidor.

1.5 Conceptos utilizados

El mutex representa exclusión mutua porque solo permite que un hilo entre al búfer a la vez. Los semáforos contadores realizan sincronización condicional: el productor depende de que exista un espacio y el consumidor depende de que exista un elemento. Por tanto, un hilo puede quedar bloqueado sin que exista un error; se desbloquea cuando el otro hilo cambia el estado del búfer.

2. Problema lectores-escritores

Instrucción. Implementar una base de datos sencilla representada mediante una variable compartida. El programa debe incluir tres hilos lectores, dos hilos escritores, la variable `numReaders`, el semáforo binario `mutex` para protegerla y el semáforo binario `wlock` para controlar el acceso a la base de datos. También debe incluir los métodos `startRead()`, `endRead()`, `startWrite()` y `endWrite()`. Varios lectores deben poder acceder simultáneamente, pero cada escritor debe tener acceso exclusivo.

2.1 Entradas

La base de datos se representa con la variable `baseDatos`, que inicia en 0. La variable `numReaders` cuenta los lectores que están dentro de la sección de lectura. El programa crea tres lectores y dos escritores; los escritores escriben los valores 100 y 200.

Cuadro 2.1: Datos iniciales del problema lectores-escritores.

Elemento	Valor inicial	Función
<code>baseDatos</code>	0	Variable compartida que leen los lectores y modifican los escritores.
<code>numReaders</code>	0	Cuenta cuántos lectores están leyendo.
<code>mutex</code>	1	Protege el contador <code>numReaders</code> .
<code>wlock</code>	1	Permite el acceso exclusivo de los escritores y del grupo de lectores.

2.2 Procesamiento

Cuando un lector inicia, ejecuta `startRead()`. Primero solicita `mutex` y aumenta `numReaders`. Si es el primer lector, solicita también `wlock`; así evita que un escritor modifique la base de datos mientras haya lectores. Después libera `mutex`. Los lectores que llegan después pueden entrar porque `wlock` ya está ocupado por el grupo de lectores.

Al terminar, cada lector ejecuta `endRead()`. Solicita `mutex`, disminuye `numReaders` y, si ya no queda ningún lector, libera `wlock`. Finalmente libera `mutex`.

Un escritor ejecuta `startWrite()` para solicitar `wlock`. Mientras lo tiene, ningún lector nuevo puede entrar y ningún otro escritor puede modificar la base de datos. Después de escribir, ejecuta `endWrite()` y libera `wlock`.

2.3 Prueba de escritorio

La siguiente secuencia muestra primero la entrada de los tres lectores y después la entrada exclusiva de los dos escritores. Los valores de `mutex` y `wlock` se muestran después de cada acción.

Cuadro 2.2: Prueba de escritorio del problema lectores-escritores.

Paso	Acción	numReaders	mutex/wlock	Estado
0	Estado inicial	0	1/1	Lectores y escritores disponibles.
1	Lector 1 ejecuta startRead()	1	1/0	Lector 1 lee; los escritores quedan bloqueados.
2	Lector 2 ejecuta startRead()	2	1/0	Lectores 1 y 2 leen al mismo tiempo.
3	Lector 3 ejecuta startRead()	3	1/0	Tres lectores están activos.
4	Lector 1 ejecuta endRead()	2	1/0	Lector 1 sale; los escritores siguen bloqueados.
5	Lector 2 ejecuta endRead()	1	1/0	Lector 2 sale; Lector 3 continúa.
6	Lector 3 ejecuta endRead()	0	1/1	Se libera wlock.
7	Escritor 1 ejecuta startWrite() y escribe 100	0	1/0	Escritor 1 tiene acceso exclusivo.
8	Escritor 1 ejecuta endWrite()	0	1/1	La base de datos queda disponible.
9	Escritor 2 escribe 200 y libera wlock	0	1/1	Escritor 2 también entra de forma exclusiva.

2.4 Salidas y evidencia

La salida depende del orden en que el sistema planifica los hilos. La captura muestra que los lectores pueden leer el mismo valor de forma concurrente, mientras que las escrituras aparecen una por una. También se observa la creación de los tres lectores y los dos escritores mediante los métodos `start()` y `join()`.

```

J Ejercicio02LectoresEscritores.java > Ejercicio02LectoresEscritores > main(String[])
3  public class Ejercicio02LectoresEscritores {
4
5      private static int baseDatos = 0;
6      private static int numReaders = 0;
7
8      private static final Semaphore mutex = new Semaphore(permits: 1);
9      private static final Semaphore wlock = new Semaphore(permits: 1);
10
11     private static void startRead() throws InterruptedException {
12         mutex.acquire();
13
14         numReaders++;
15
16         if (numReaders == 1) {
17             wlock.acquire();
18         }
19
20         mutex.release();
21     }
22
23     private static void endRead() throws InterruptedException {
24         mutex.acquire();
25
26         numReaders--;
27
28         if (numReaders == 0) {
29             wlock.release();
30         }
31
32         mutex.release();
33     }
34
35     private static void startWrite() throws InterruptedException {
36         wlock.acquire();
37     }
38
39     private static void endWrite() {
40         wlock.release();
41     }
42
43     private static void leer() {
44         try {
45             startRead();

```

Figura 2.1: Código Java de los lectores y escritores, parte superior.

```
40     wlock.release();
41 }
42
43 private static void leer() {
44     try {
45         startRead();
46
47         String nombre = Thread.currentThread().getName();
48         System.out.println(nombre + " lee: " + baseDatos);
49
50         Thread.sleep(500);
51
52         endRead();
53     } catch (InterruptedException e) {
54         Thread.currentThread().interrupt();
55     }
56 }
57
58
59 private static void escribir(int valor) {
60     try {
61         startWrite();
62
63         String nombre = Thread.currentThread().getName();
64
65         baseDatos = valor;
66         System.out.println(nombre + " escribe: " + baseDatos);
67
68         Thread.sleep(500);
69
70         endWrite();
71     } catch (InterruptedException e) {
72         Thread.currentThread().interrupt();
73     }
74 }
75
76 }
```

Figura 2.2: Código Java de los lectores y escritores, parte inferior.

```

Ejercicio02LectoresEscritores.java > Ejercicio02LectoresEscritores > main(String[])
3 public class Ejercicio02LectoresEscritores {
77     public static void main(String[] args) throws InterruptedException {
78
79         Thread lector1 = new Thread(() -> leer(), name: "Lector 1");
80         Thread lector2 = new Thread(() -> leer(), name: "Lector 2");
81         Thread lector3 = new Thread(() -> leer(), name: "Lector 3");
82
83         Thread escritor1 = new Thread(() -> escribir(valor: 100), name: "Escritor 1");
84         Thread escritor2 = new Thread(() -> escribir(valor: 200), name: "Escritor 2");
85
86         lector1.start();
87         escritor1.start();
88         lector2.start();
89         lector3.start();
90         escritor2.start();
91
92         lector1.join();
93         lector2.join();
94         lector3.join();
95         escritor1.join();
96         escritor2.join();
97     }
98 }

```

Execution output (multiple instances):

```

(heri) ./concurrente -I' main ?
Lector 1 lee: 0
Lector 2 lee: 0
Lector 3 lee: 0
Escritor 1 escribe: 100
Escritor 2 escribe: 200

(heri) ./concurrente -I' main ?
Escritor 1 escribe: 100
Escritor 2 escribe: 200
Lector 1 lee: 200
Lector 2 lee: 200
Lector 3 lee: 200

(heri) ./concurrente -I' main ?
Escritor 1 escribe: 100
Escritor 2 escribe: 200
Lector 1 lee: 100
Lector 2 lee: 100
Lector 3 lee: 100
Escritor 2 escribe: 200

(heri) ./concurrente -I' main ?
Escritor 1 escribe: 100
Escritor 2 escribe: 200
Lector 1 lee: 0
Lector 2 lee: 0
Lector 3 lee: 0
Escritor 1 escribe: 100
Escritor 2 escribe: 200

(heri) ./concurrente -I' main ?
Escritor 1 escribe: 100
Escritor 2 escribe: 200
Lector 1 lee: 200
Lector 2 lee: 200
Lector 3 lee: 200

```

Figura 2.3: Creación de los hilos y resultados de las ejecuciones del problema lectores-escritores.

2.5 Conceptos utilizados

En este problema, mutex protege una variable de control: numReaders. El semáforo wlock protege la base de datos. La exclusión mutua se observa cuando un escritor modifica la base de datos sin que otro escritor o un lector pueda hacerlo al mismo tiempo.

La sincronización condicional aparece cuando un hilo debe esperar una condición. Un escritor espera mientras exista al menos un lector. El primer lector toma wlock y el último lector lo libera. Así, varios lectores pueden compartir la lectura, pero las escrituras permanecen aisladas.

3. Conclusión

El problema productor-consumidor utiliza un semáforo binario y dos semáforos contadores. El semáforo binario protege el búfer, mientras que los contadores indican si el productor puede depositar o si el consumidor puede retirar. La espera ocurre cuando el búfer está lleno o vacío.

El problema lectores-escritores utiliza un semáforo para proteger numReaders y otro para controlar la base de datos. Los lectores pueden entrar juntos, pero un escritor necesita acceso exclusivo. En ambos programas los semáforos evitan condiciones de carrera y bloquean los hilos solo cuando no se cumple la condición necesaria para continuar.

La exclusión mutua significa que una sección crítica solo puede ser utilizada por un hilo a la vez. La sincronización condicional establece el momento en que un hilo puede continuar, por ejemplo cuando aparece un espacio libre, un elemento disponible o cuando termina el último lector.

Bibliografía

- [1] V. G. Morales Murillo, “Semáforo, semáforo binario y semáforo contador,” 2026, diapositivas del curso Programación Concurrente, Universidad de las Américas Puebla.